Pack 01 · Speed physics

Why decode is limited by memory, prefill by math, and how batching changes both. Lessons 01–04, 06, 09, 15, 16, 24, 25, 28, 44, 59–61. Answer each question out loud in about a minute, then check the card.

Q1 · "Can we advertise 300 tokens/s?" PRIORITY

Lessons: 02, 06, 16, 24

> **INTERVIEWER**
>
> We serve Llama 3 8B in BF16 on one H100 with a single user online. The PM wants to advertise 300 tokens/s streaming, and a teammate's benchmark already shows 2,000 tokens/s. Can we promise it?

60-second answer

- No. At batch 1, every decode step reads **all the weights** from HBM once, so the step can't be faster than bytes ÷ bandwidth.
- 16 GB ÷ 3.35 TB/s ≈ **4.8 ms per token**, a ceiling of ≈ **200 tokens/s**; real kernels land around 150–180.
- The 2,000 tokens/s benchmark is broken. The usual bugs: **no GPU synchronize** before stopping the timer (kernels run asynchronously), and **counting prompt tokens as output**.
- To go faster per user: fewer bytes (quantization), or more tokens per weight read (speculative decoding).

Key numbers: decode floor = weight bytes ÷ bandwidth = $16 \times 10^9 \div 3.35 \times 10^{12} \approx 4.8$ ms.

Likely follow-up: "Is our 70B FP8 baseline healthy at 14 ms TPOT on two H100s with tensor parallelism?" The weights are split, so the bandwidths add: 70 GB ÷ 6.7 TB/s ≈ 10.4 ms floor. MBU = 10.4 ÷ 14 ≈ 74%, which is healthy.

> **TRAP**
>
> I once said the ceiling comes from TFLOPS. Decode is limited by memory bandwidth. With tensor parallelism, add the GPUs' bandwidths.

Q2 · Why is a 500-token prefill only about 2× one decode step? PRIORITY

Lessons: 04, 15, 16, 44

> **INTERVIEWER**
>
> Prefill of a 500-token prompt takes about 8 ms; one decode token takes about 4.8 ms. Why so close, and how long do 30 streaming users freeze when someone pastes an 8,000-token document?

60-second answer

- Each phase is limited by a different resource: time ≈ **max(FLOPs ÷ peak FLOPS, bytes ÷ bandwidth)**.
- **Decode** does little math per byte, so it's limited by reading the weights: 4.8 ms.
- **Prefill** processes all prompt tokens in parallel, reading the weights once, so it's limited by math: $2 \times 8 \times 10^9 \times 500 \approx 8 \times 10^{12}$ FLOP ÷ 989 TFLOPS ≈ 8 ms.
- An 8,000-token prefill is $2 \times 8 \times 10^9 \times 8{,}000 \approx 1.3 \times 10^{14}$ FLOP ÷ 989 TFLOPS ≈ **130 ms**: every stream freezes for about **27 decode steps**. Chunked prefill fixes it (pack 03).

Key numbers: forward-pass FLOPs ≈ 2 × parameters × tokens; H100 dense BF16 ≈ 989 TFLOPS.

Likely follow-up: "What's the ridge point?" 989 TFLOPS ÷ 3.35 TB/s ≈ 295 FLOP per byte. Below it you're memory-bound, above it compute-bound.

> **TRAP**
>
> I divided bytes by FLOPS. That isn't a time. Bytes go over bandwidth, FLOPs over FLOPS, and prefill cost must scale with the prompt length.

Q3 · "32 users means 32× slower, right?" PRIORITY

Lessons: 02, 16, 25, 28

> **INTERVIEWER**
>
> A PM says serving 32 users at once makes each user 32× slower. Llama 3 8B in BF16 on an H100; each user holds about 1,000 tokens of KV at 128 KiB per token. What really happens to per-user speed and total throughput?

60-second answer

- **Alone:** each step reads the 16 GB of weights, 16 GB ÷ 3.35 TB/s ≈ **4.8 ms**, and makes 1 token: ≈ **200 tokens/s**.
- **Batched:** the **16 GB of weights is read once per step for everyone**. Only each user's KV cache is extra: 32 × 1,000 × 128 KB ≈ 4.1 GB. Step ≈ (16 + 4.1) GB ÷ 3.35 TB/s ≈ **6.0 ms**, and that one step makes **32 tokens**, one per user.
- **Per user:** still 1 token per step, but the step is 6.0 ÷ 4.8 = 1.25× longer, so each user is about **25% slower** (≈ 167 tokens/s instead of 200).
- **Whole GPU:** 32 tokens every 6 ms = 32 ÷ 0.006 s ≈ **5,300 tokens/s**, about **25×** the 200 alone. Not the full 32× because the step got 1.25× longer: 32 ÷ 1.25 ≈ 25.6.
- The cost of batching is **KV memory**, which grows with batch × context.

Key numbers: step ≈ (weights + batch × context × KV per token) ÷ bandwidth; total tokens/s = batch ÷ step; per user = 1 ÷ step.

Likely follow-up: "Our FastAPI server calls `generate()` and 20 users wait 40 s, with the GPU's math mostly idle. Bigger GPU?" No: requests run one at a time, so the queue is the problem. Continuous batching lets all 20 share each weight read.

> **TRAP**
>
> I added one user's KV instead of 32 users'. If the question says 32 users, a 32 must appear in the byte sum. Check ms → tokens/s against batch 1 (4.8 ms ≈ 200).

Q4 · When does decode become compute-bound? SOLID

Lessons: 02, 04, 61

> **INTERVIEWER**
>
> With short contexts, you keep raising the decode batch. At what batch does decode flip from memory-bound to compute-bound, and why doesn't that happen as early in practice?

60-second answer

- Per token, BF16 decode does 2 FLOP per parameter and reads 2 bytes per parameter **shared across the batch**, so intensity ≈ **batch size** FLOP per byte.
- It crosses the H100 ridge (≈ 295 FLOP per byte) at a batch of **about 300**.
- In practice the **KV cache** read grows with batch × context, and attention's KV reads are never shared, so long contexts keep decode memory-bound much longer.

Key numbers: ridge = peak FLOPS ÷ bandwidth ≈ 989 ÷ 3.35 ≈ 295.

Likely follow-up: "So what does batching buy once you're compute-bound?" Nothing more per weight read: throughput flattens and per-user latency rises. That's the saturation wall in a concurrency sweep.

> **TRAP**
>
> say what the ratio is (FLOP per byte) and that the weight bytes are shared by the batch while KV bytes are not.

Q5 · "GPU-util is 100%, we need more GPUs" PRIORITY

Lessons: 09, 16, 59, 60

> **INTERVIEWER**
>
> One user chats with Llama 3 8B on an H100. The dashboard shows GPU-util 100%, power 350 W of a 700 W cap, and 6 ms per decode token. The manager wants more GPUs. Is it maxed out?

60-second answer

- **GPU-util** only says a kernel was running in each sampling window (duty cycle). It says nothing about how much of the chip is busy.
- Measure **MBU**: (16 GB ÷ 6 ms) ÷ 3.35 TB/s ≈ 2.7 ÷ 3.35 ≈ **80%** of bandwidth. Compute (MFU) is nearly idle.
- **Half power is expected** for memory-bound decode: the tensor cores are mostly waiting for data.
- The fix is **more users per GPU** (batching), not more GPUs.

Key numbers: MBU = achieved bandwidth ÷ peak bandwidth = floor ÷ measured step.

Likely follow-up: "What would you put on the dashboard instead?" MBU and MFU, tokens/s, KV cache usage, running and waiting requests, TTFT and TPOT percentiles.

> **TRAP**
> I read half power as "half the units idle, so the kernels need work", and divided by the ridge instead of peak bandwidth.

Q6 · What does a million tokens cost? PRIORITY

Lessons: 01, 02

> **INTERVIEWER**
> An H100 rents for $3 an hour. What do a million output tokens cost at one user (~200 tokens/s) and at 64 users batched (~4,500 tokens/s)?

60-second answer

- Tokens per hour = tokens/s × 3,600.
- One user: 200 × 3,600 = 720,000 per hour, so 1 M tokens takes ≈ 1.4 h ≈ **$4.17**.
- 64 users: 4,500 × 3,600 ≈ 16 M per hour, so ≈ **$0.19 per million**, about 22× cheaper.
- Batching is the main cost lever: the GPU-hour costs the same either way.

Key numbers: cost per 1 M = $/hour ÷ (tokens/s × 3,600) × 10^6.

Likely follow-up: "Where does the 64-user step time come from?" (16 GB + 64 × 4,096 × 128 KB ≈ 48 GB) ÷ 3.35 TB/s ≈ 14 ms, so ≈ 71 tokens/s per user and ≈ 4,500 total.

> **TRAP**
> I divided dollars by hours instead of multiplying, and slipped a power of ten (10^6 ÷ 200 is 5,000 s, not 500).

Q7 · Kernel launches and Amdahl PRIORITY

Lessons: 03, 04, 09, 60

> **INTERVIEWER**
>
> A decode step launches about 300 kernels at about 4 μs of CPU time each, against an ideal step of 4.8 ms. How much does that cost, and how do you cut it? Separately, a teammate made attention (30% of the step) 3× faster and claims decode is now 3× faster. What's the real speedup?

60-second answer

- Launches: 300 × 4 μs = 1.2 ms, **25%** of a 4.8 ms step. Fix with **CUDA graphs** (record the step once, replay it with one launch) and **kernel fusion**.
- Amdahl: new time = 0.7 + 0.3 ÷ 3 = 0.8, so the speedup is 1 ÷ 0.8 = **1.25×**, not 3×.
- Even if attention were free, the ceiling is 1 ÷ 0.7 ≈ **1.43×**.
- Lesson: speed up the part that dominates the time.

Key numbers: overall speedup = 1 ÷ ((1 − p) + p ÷ s).

Likely follow-up: "Two optimizations: attention 30% made 3×, MLP 50% made 2×?" New time = 0.2 + 0.1 + 0.25 = 0.55, so 1.8×. Speedups never add or multiply.

> **TRAP**
>
> I offered "fewer bytes" as a fix for launch overhead (that speeds up the bandwidth part), and gave the ceiling as "70%" instead of a ratio of times.

Build it yourself

- `exercises/03_04_amdahl.py`: an Amdahl calculator for several non-overlapping optimizations. It drills time accounting instead of adding speedups. Run `uv run exercises/03_04_amdahl.py`.